Progettazione ad oggetti (OOD) e pattern di progetto

- **L'Object Oriented Analysis and Design(OOAD) si compone di quattro componenti principali**
  - Il modello del sistema (PDC)
    - Deriva dall'OOA
    - Descrive i dati intrinsechi che caratterizzano il sistema
  - L'Interazione Utente (HIC)
    - Indica come gli utenti interagiscono con il sistema e con le applicazioni che lo compongono
    - Comprende il livello applicativo
      - Ossia le applicazioni che manipolano i dati del modello interagendo con l'utente e/o con i database
    - Comprende le GUI
      - Ossia le finestre e le parti che compongono l' interfaccia utente delle applicazioni
  - La Gestione dei dati permanenti (DMC)
    - Gestisce la memorizzazione degli oggetti in modo permanente
  - L'interazione con sistemi esterni e con la rete (SIC)
    - Incorpora le funzionalità necessarie per far comunicare il sistema con sistemi e dispositivi esterni e le reti
- **Pattern di progettazione Object Oriented**

Presented with xmind

Pattern di progettazione Object Oriented

Definizione

- Sono schemi di oggetti utilizzati per risolvere una determinata classe di problemi
- Gli oggetti all'interno di un pattern hanno responsabilità specifiche e interazioni reciproche
- I pattern sono blocchi per costruire programmi, molto utili per ottenere modelli ad oggetti efficaci.
- Essi sono stati introdotti per primi nel libro "Design Patterns" nel 1995

Esistono tre principali pattern di progettazione

- Pattern creazionali — Si riferiscono alla creazione degli oggetti
- Pattern strutturali — Si riferiscono alla struttura dati degli oggetti
- Pattern comportamentali — Si riferiscono alle funzionalità degli oggetti
- In totale i pattern del libro di Gamma sono 23

La documentazione di un pattern si compone di quattro elementi

- Il nome
  - Identifica il pattern
  - Deve essere il più possibile descrittivo in una parola o due
- Il problema
  - Descrive quando usare il pattern e, se del caso, i vincoli di applicabilità
  - Comprende lo scopo e l'utilizzo del pattern
- La soluzione
  - Descrive gli elementi che costituiscono il pattern (partecipanti), la struttura, le collaborazioni tra oggetti ed eventualmente il funzionamento
- Le conseguenze
  - Sono il risultato effettivo e gli effetti collaterali dovuti all'applicazione del pattern
  - Sono fondamentali per valutare le alternative di progetto

Presented with xmind

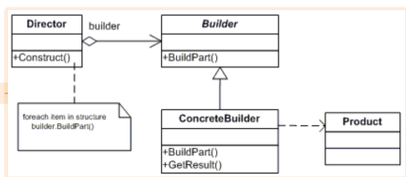
Pattern creazionali

Definizione

- Sono un tipo di pattern di progettazione che si concentrano sulla creazione e l' inizializzazione di oggetti
- Lo scopo principale è quello di ridurre l' accoppiamento tra le classi software
  - In particolare, il disaccoppiamento della creazione degli oggetti dalla specifica diretta della loro classe
  - Invece di creare gli oggetti direttamente dentro la classe viene utilizzato uno schema predefinito che determina il modo in cui vengono creati e inizializzati
- Viene applicato l'approccio di Dependency Inversion per rendere il software più facile da modificare in caso di aggiunta di oggetti
- Si basano su gerarchie di classi e sul polimorfismo della programmazione ad oggetti
  - Rende il software più modulare e mantenibile.

Pattern Builder (o Generator Pattern)

- È un pattern creazionale in cui un oggetto complesso viene costruito utilizzando un produttore separato dall'oggetto stesso
- Il produttore è noto come il Builder
  - Questo riceve i parametri necessari e produce l'oggetto complesso attraverso una serie di passaggi.
- **Scopo**
  - Lo scopo è quello di separare la costruzione di un oggetto complesso dalla sua rappresentazione in modo che lo stesso processo possa creare differenti rappresentazioni
- **Utilizzo**
  - L'algoritmo per creare un oggetto complesso non deve dipendere dalle parti specifiche che lo compongono e da come sono messe insieme
  - Il processo di costruzione deve consentire diverse rappresentazioni dell'oggetto da costruire
- **I partecipanti principali sono**
  - **Il Costruttore (classe astratta)**
    - Specifica un'interfaccia astratta per creare le parti dell'oggetto Prodotto
  - **Il Costruttore Concreto**
    - Costruisce e mette insieme le parti del prodotto, implementando l'interfaccia del costruttore,
    - Definisce e tiene traccia della rappresentazione dal lui generata
    - Fornisce un'interfaccia per accedere al prodotto
  - **Il Direttore**
    - Costruisce un oggetto utilizzando l' interfaccia del Costruttore
  - **Il Prodotto**
    - È l'oggetto complesso in costruzione
    - Comprende le classi che definiscono le parti costituenti, con le interfacce per comporle insieme
- **Struttura**

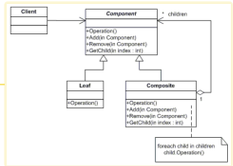


- **Collaborazioni**
  - Il Cliente crea l'oggetto Direttore e lo configura con il Costruttore Concreto
  - La decisione di quale Costruttore Concreto usare è presa a runtime
  - Il Direttore comanda al costruttore concreto di creare le parti opportune, usando l' interfaccia del costruttore astratto
  - Il Costruttore Concreto esegue gli ordini del Direttore e aggiunge parti al Prodotto
  - Il Cliente ottiene il Prodotto dal Costruttore
- **Conseguenze**
  - La rappresentazione interna del prodotto può variare, senza che il Direttore o il Cliente ne siano influenzati
  - Isola il codice che costruisce il Prodotto, incapsulando la costruzione e la rappresentazione di un oggetto complesso
  - Consente un maggior controllo del processo di costruzione fatto passo-passo

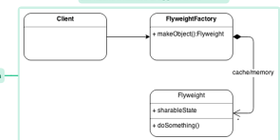
Presented with xmind

Pattern strutturali

- **Definizione**
  - Forniscono schemi su come comporre classi ed oggetti per formare strutture complesse
  - **I pattern strutturali di istanza sono i più utilizzati**
    - Descrivono metodi per comporre oggetti in modo da ottenere nuove funzionalità
- **Scopo**
  - Il disaccoppiamento dell'accesso agli oggetti dalla loro specifica implementazione
  - La possibilità di aggiungere caratteristiche in modo incrementale
- **Pattern Composito (Composite)**
  - **Scopo**
    - Rappresentare in modo uniforme oggetti che contengono altri oggetti, sia semplici che a loro volta composti
  - **Utilizzo**
    - Quando si vogliono rappresentare oggetti che contengono gerarchie di altri oggetti, in modo che l'interfaccia degli oggetti semplici (foglie) e di quelli composti sia uniforme
  - **Partecipanti**
    - **Componente (Classe astratta)**
      - Definisce l'interfaccia degli oggetti di una composizione
      - Implementa il comportamento di default di tutte le loro classi, se esiste
      - Definisce l'interfaccia per gestire i componenti "figli" (quelli contenuti in un " padre")
    - **Foglia**
      - Rappresenta gli oggetti foglia, quelli che non contengono altri oggetti
      - Definisce il comportamento degli oggetti primitivi della composizione
    - **Composito**
      - Definisce il comportamento degli oggetti che ne contengono altri
      - Contiene oggetti "figli" ed implementa la gestione di tali figli
    - **Cliente**
      - Manipola gli oggetti della composizione usando l'interfaccia Componente
  - **Struttura**



  - **Collaborazioni**
    - I Clienti usano l'interfaccia della classe Componente per interagire con li oggetti della struttura complessa
      - Se il ricevente è una foglia, esegue direttamente la richiesta
      - Se il ricevente è un composito, la passa ai suoi Componenti, eventualmente eseguendo operazioni aggiuntive
  - **Conseguenze**
    - Un Composito permette di definire gerarchie di classi composte da oggetti primitivi che si possono raggruppare in oggetti complessi, a vari livelli
    - I Clienti possono trattare oggetti composti e oggetti primitivi in modo uniforme, e sono quindi liberati dalla gestione esplicita di tali strutture
    - È più facile aggiungere nuovi oggetti, Foglia o Composti
- **Pattern Flyweight (Peso mosca)**
  - **Scopo**
    - Rappresentare in un solo posto oggetti identici, riferiti da molti altri oggetti in un sistema
  - **Utilizzo**
    - Quando si utilizzano molti oggetti identici, che se duplicati occuperebbero una quantità esagerata di memoria
  - **Partecipanti**
    - Cliente
      - Oggetto che crea e richiede uso di un oggetto "flyweight"
    - FlyweightFactory
      - Crea gli oggetti "flyweight", differenziando la creazione del primo di questi da quella di tutti gli oggetti identici successivi
    - Flyweight
      - Si riferisce agli oggetti che vogliamo riusare per ridurre il numero di oggetti creati
  - **Struttura**



  - **Collaborazioni**
    - I clienti usano l'interfaccia della classe FlyweightFactory tutte le volte che vogliono creare un oggetto di cui ci possono essere molte copie identiche
      - Se l'oggetto non esiste, FlyweightFactory lo crea e lo inserisce in una struttura dati e lo restituisce
      - Se l'oggetto già esiste, la FlyweightFactory lo ritrova e lo restituisce, senza crearne uno nuovo
  - **Conseguenze**
    - Un Flyweight permette di risparmiare spazio di memoria in proporzione al numero di istanze identiche utilizzate
    - I Flyweight possono introdurre costi di esecuzione associati alla ricerca degli oggetti esistenti, e al calcolo dello stato dipendente dallo specifico oggetto (stato estrinseco)
    - Il pattern Flyweigth è spesso combinato con il pattern Composite per rappresentare una struttura gerarchica come un grafo con nodi foglia condivisi

Presented with xmind

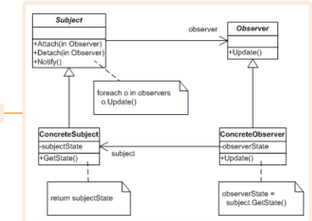
Pattern comportamentali

Definizione

- Forniscono schemi che riguardano gli algoritmi e la suddivisione di responsabilità di oggetti
- Anche i pattern comportamentali possono essere di classe o d'istanza
  - Quelli d'istanza sono i più comuni
  - I pattern comportamentali d'istanza descrivono modi che usano la composizione di oggetti in modo da farli cooperare per ottenere un comportamento complesso
- Tendono ad avere la massima distribuzione delle responsabilità e la massima uniformità nelle interfacce
- Il loro scopo principale è il disaccoppiamento degli oggetti cooperanti, in modo da poter riutilizzare facilmente i singoli oggetti e da facilitare le modifiche del software

Pattern Osservatore (observer)
o Publish-Subscribe
o Model-View-Controller

- **Scopo**
  - Definire una dipendenza da uno a molti tra oggetti in modo che quando un oggetto cambia stato tutti i suoi dipendenti ne siano automaticamente notificati
- **Utilizzo**
  - Quando un'astrazione ha due aspetti indipendenti, da incapsulare in oggetti separati da poter variare e riusare separatamente
  - Quando la modifica di un oggetto implica modifiche ad altri oggetti non noti a priori
  - Se un oggetto deve notificare informazioni ad altri oggetti senza doverli conoscere direttamente, mantenendo quindi un' accoppiamento debole
- **Partecipanti**
  - **Soggetto (classe astratta)**
    - Conosce i propri Osservatori, che possono essere in numero qualsiasi
    - Fornisce un'interfaccia per aggiungere e togliere Osservatori da sé stesso
  - **Osservatore (classe astratta)**
    - Definisce un'interfaccia per aggiornare gli oggetti cui vanno notificati i cambiamenti
  - **SoggettoConcreto**
    - Memorizza i dati che interessano i propri Osservatori Concreti
    - Notifica i propri Osservatori concreti quando cambia stato
  - **OsservatoreConcreto**
    - Ha un riferimento ad un Soggetto Concreto
    - Ha uno stato che deve essere coerente con quello del proprio Soggetto Concreto
    - Implementa l'interfaccia dell'Osservatore per mantenere il proprio stato coerente con quello del proprio Soggetto Concreto
- **Struttura**



- **Collaborazioni**
  - Un Soggetto Concreto notifica i propri osservatori quando avviene un cambiamento nel proprio stato
  - Dopo essere stato notificato del cambiamento, un Osservatore Concreto può richiedere informazioni al proprio soggetto per riconciliare il proprio stato con esso
- **Conseguenze**
  - L'accoppiamento tra Soggetto e Osservatore è astratto: il Soggetto sa solo che ha una lista di Osservatori, e che deve notificare loro quando cambia. La classe specifica degli Osservatori e del soggetto non ha importanza
  - La notifica di cambiamento non deve specificare il ricevente ma è inviata automaticamente a tutti gli osservatori attaccati al Soggetto

Presented with xmind